Chapitre 1 — Environnement de développement et premier programme

Exercices : `E01_Environnement_premier_programme`

Objectifs

- distinguer éditeur, compilateur et débogueur ;
- compiler et exécuter un programme C++ ;
- lire un message du compilateur pour localiser une erreur ;
- distinguer erreur de compilation, avertissement et erreur d'exécution ;
- suivre l'exécution d'un programme pas à pas avec le débogueur.

Sommaire

- Chapitre 1 — Environnement de développement et premier programme
  - Objectifs
  - Sommaire
  - Conventions du cours
  - 1. Les outils du développeur
  - 2. Du code source à l'exécutable
  - 3. Structure d'un programme C++
  - 4. Compiler et exécuter avec g++
  - 5. Lire un message du compilateur
  - 6. Déboguer avec Visual Studio Code
    - 6.1 Préparer le débogage
    - 6.2 Exécuter pas à pas
  - Erreurs fréquentes
  - Synthèse

Conventions du cours

Environnement : Visual Studio Code et le compilateur g++, norme **C++17**.

Dans tous les documents :

- les variables sont initialisées dès leur déclaration, de préférence avec `{}` (chapitre 2) ;
- les textes affichés sont écrits sans accents, car certaines consoles, notamment sous Windows, les affichent mal ;
- un bloc de code qui contient `main()` est un **programme complet** qui compile tel quel ; les autres blocs sont des **extraits**, à placer dans un programme ;
- un bloc sur fond sombre, annoncé par « Résultat », montre ce que le programme affiche.

1. Les outils du développeur

Outil	Rôle
Éditeur de code	Écrire et modifier le code source
Compilateur	Traduire le code source en programme exécutable
Débogueur	Exécuter le programme pas à pas pour rechercher les erreurs

Visual Studio Code est l'éditeur. Le compilateur g++ et le débogueur gdb sont des programmes séparés, que Visual Studio Code sait piloter.

2. Du code source à l'exécutable

Un programme est écrit dans un **fichier source** d'extension `.cpp`. On l'écrit, on le compile, on corrige les erreurs signalées, on l'exécute, puis on le teste.

Type	Moment	Conséquence
Erreur de compilation	À la compilation	Aucun exécutable produit (ex. : point-virgule oublié)
Avertissement (*warning*)	À la compilation	Exécutable produit, mais risque signalé (ex. : variable utilisée sans valeur)
Erreur d'exécution ou de logique	À l'exécution	Plantage ou résultat faux

> **Remarque :** un avertissement signale presque toujours un vrai problème. On le corrige comme une erreur.

> **Attention :** une erreur de logique ne produit **aucun** message. Le programme compile, s'exécute… et donne un résultat faux. Seuls les tests la révèlent.

3. Structure d'un programme C++

On commence par un programme simple, `prog_1.cpp`, qui affiche un message :

```cpp
#include <iostream>

int main() {
    std::cout << "Bonjour !" << '\n';
    return 0;
}
```

Résultat :

```
Bonjour !
```

- `#include <iostream>` donne accès aux entrées-sorties standard (`std::cout`, `std::cin`).
- `int main()` est la fonction principale : l'exécution commence par elle.

- `{ }` délimitent un **bloc** ; chaque instruction simple se termine par `;`.
- `std::cout << …` envoie des données vers la console ; `'\n'` insère un retour à la ligne.
- Un texte s'écrit entre **guillemets doubles** (`"Bonjour !"`) ; les **apostrophes** entourent **un seul caractère** (`'\n'`).
- `return 0;` indique une fin normale.
- `std::` désigne l'**espace de noms standard**, qui regroupe les outils de la bibliothèque standard.

Les **commentaires** ne sont pas exécutés :

```
// Commentaire sur une ligne

/* Commentaire
   sur plusieurs lignes */
```

> **Conseil :** un bon commentaire explique **pourquoi**, pas ce que fait une instruction évidente.

4. Compiler et exécuter avec g++

Dans le terminal de Visual Studio Code, placé dans le dossier du fichier :

```
g++ -std=c++17 -Wall -Wextra prog_1.cpp -o prog_1
./prog_1
```

Élément	Signification
`g++`	Appel du compilateur
`-std=c++17`	Norme C++17
`-Wall -Wextra`	Principaux avertissements activés
`prog_1.cpp`	Fichier source
`-o prog_1`	Nom de l'exécutable produit

> **Remarque :** la commande minimale `g++ prog_1.cpp -o prog_1` fonctionne aussi, mais elle n'active pas les avertissements et utilise la norme par défaut du compilateur. Dans ce cours, on utilise **toujours** la commande complète.

Sous Windows, l'exécutable s'appelle `prog_1.exe` et se lance avec `.\prog_1.exe`.

> **Attention :** après chaque modification du fichier source, il faut **recompiler**. Sinon, on exécute l'ancienne version du programme.

5. Lire un message du compilateur

Chaque message de g++ suit le même modèle :

```
fichier:ligne:colonne: type: message
```

Exemple : on a oublié la ligne `#include <iostream>` dans un fichier `demo.cpp`. La compilation affiche :

```
demo.cpp:2:10: error: 'cout' is not a member of 'std'
    2 |     std::cout << "Demarrage" << '\n';
      |          ^~~~
demo.cpp:1:1: note: 'std::cout' is defined in header '<iostream>'; did you forget to
'#include <iostream>'?
```

Élément du message	Ici	Ce qu'il indique
Fichier, ligne, colonne	`demo.cpp:2:10`	L'endroit où le compilateur a détecté le problème
Type	`error`	`error` : pas d'exécutable ; `warning` : avertissement ; `note` : information complémentaire
Message	`'cout' is not a member of 'std'`	La description du problème
Ligne de code et `^~~~`	sous `cout`	Le passage exact visé

> **Méthode :** on traite les messages **dans l'ordre**, en commençant par le **premier** : une seule erreur peut en provoquer plusieurs autres. On lit la ligne indiquée, **puis la ligne précédente**, car le compilateur détecte parfois un oubli seulement en lisant la suite. Les `note` donnent souvent la correction.

6. Déboguer avec Visual Studio Code

Le **débogueur** exécute le programme sous contrôle : il l'arrête sur une ligne choisie, puis le fait avancer instruction par instruction. On peut ainsi vérifier l'ordre d'exécution et, à partir du chapitre 2, la valeur des variables.

6.1 Préparer le débogage

- Installer l'extension **C/C++** de Microsoft dans Visual Studio Code.
- Le débogueur **gdb** doit être installé avec g++ (il est fourni avec la plupart des installations ; sinon, l'installer).
- Le programme doit être compilé avec l'option `-g`, qui ajoute les informations de débogage :

```
g++ -std=c++17 -Wall -Wextra -g prog_1.cpp -o prog_1
```

Lors du premier lancement (§6.2), Visual Studio Code propose une configuration : choisir « **C/C++: g++ build and debug active file** ». Il crée alors un fichier `.vscode/tasks.json` qui compile avec `-g`.

> **Conseil :** pour garder les avertissements pendant le débogage, ajouter `"-std=c++17"`, `"-Wall"` et `"-Wextra"` dans la liste `"args"` du fichier `.vscode/tasks.json`.

6.2 Exécuter pas à pas

1. Cliquer dans la marge, à gauche du numéro d'une ligne : un **point d'arrêt** (rond rouge) apparaît. Raccourci : `F9`.
2. Lancer le débogage : `F5`. Le programme s'arrête sur le point d'arrêt.
3. Faire avancer le programme avec les commandes ci-dessous.

Commande	Raccourci	Effet
Continuer	`F5`	Reprend jusqu'au point d'arrêt suivant (ou la fin)
Pas à pas principal	`F10`	Exécute la ligne courante, puis s'arrête à la suivante
Pas à pas détaillé	`F11`	Entre dans la fonction appelée (utile au chapitre 5)
Arrêter	`Maj+F5`	Termine le débogage

> **À retenir :** la ligne surlignée en jaune est la **prochaine** ligne à exécuter : elle n'a **pas encore** été exécutée.

Pendant une pause, le panneau **Variables** affiche la valeur de chaque variable du programme ; il servira dès le chapitre 2.

Erreurs fréquentes

Erreur	Symptôme	Correction
Point-virgule oublié	`expected ';' before …` : g++ pointe la fin de la ligne fautive et cite le premier mot de la ligne suivante	Lire la ligne indiquée **et** la précédente
`#include` oublié	`'cout' is not a member of 'std'`	Ajouter `#include <iostream>`
Exécuter sans recompiler	Les modifications n'ont aucun effet	Recompiler après chaque modification
Avertissement ignoré	Programme qui « marche » mais donne parfois un résultat faux	Corriger tous les avertissements
Débogage sans `-g`	Les points d'arrêt sont ignorés	Compiler avec `-g` (ou lancer par `F5` avec la configuration g++)

Synthèse

- On compile avec `g++ -std=c++17 -Wall -Wextra fichier.cpp -o programme`, et on corrige **aussi** les avertissements.
- On recompile après chaque modification.
- L'exécution commence par `main()` ; chaque instruction se termine par `;`.
- Un message g++ indique `fichier:ligne:colonne: type: message` ; on traite le **premier** message d'abord, en lisant aussi la ligne précédente.
- Erreur de compilation : pas d'exécutable. Avertissement : exécutable, mais risque. Erreur de logique : aucun message, résultat faux.

- Le débogueur (`F9`, `F5`, `F10`) arrête le programme et le fait avancer ligne par ligne ; la ligne surlignée n'est pas encore exécutée.